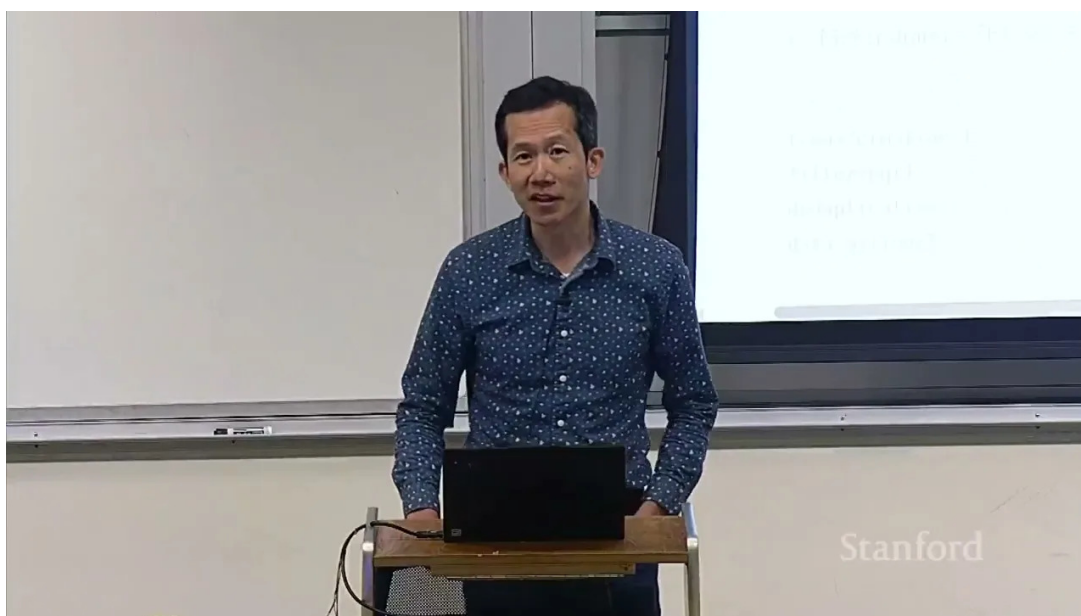


Stanford CS336 第 14 讲：数据处理与混合

标准中文图文课程笔记

五道口纳什 & Codex

2026 年 8 月 30 日



频道：Stanford Online **发布日期：**2026 年 5 月 19 日

时长：1:24:46 **链接：**<https://youtu.be/5sxHosTLPF8>

字幕：YouTube 平台人工英文字幕 en-US；未使用 Whisper。

目录

1 从原始来源到训练 token	3
1.1 本章小结	3
2 转换：把复杂文档还原成阅读顺序	4
2.1 本章小结	4
3 过滤：数量与质量的非单调关系	5
3.1 本章小结	5
4 重复：从完全相同到模板近似	6
4.1 本章小结	7
5 精确去重与哈希分组	8
5.1 本章小结	8
6 MinHash：近似 Jaccard 相似度	9
6.1 本章小结	9
7 LSH：只生成可能相似的候选对	10
7.1 本章小结	11
8 数据混合：训练分布是一组权重	12
8.1 本章小结	12
9 epoching：小而优的数据也会被耗尽	13
9.1 本章小结	13
10 回归式混合：用代理模型搜索配方	14
10.1 本章小结	15
11 合成数据：从环境和教师收集响应	16
11.1 本章小结	16
12 代码智能体任务构造	17
12.1 本章小结	18
13 教师数据的偏差与蒸馏边界	19
13.1 本章小结	19
14 端到端数据协议	20
14.1 本章小结	20

15 总结与延伸	21
15.1 进一步推论	21
15.2 建议练习	21

1 从原始来源到训练 token

上一讲回答“数据从哪里来”，本讲回答“拿到之后怎样变成可训练分布”。流水线包含转换、过滤、去重、混合，以及利用强模型构造中期/后训练数据。

```
12  Lecture 14: Data II
13  Last lecture:
14  • Live service (e.g., GitHub) → dump/crawl (e.g., GitHub Archive) → processed data (e.g., The Stack)
15  • Considerations: terms of service, copyright (licenses or fair use)
16
17  This lecture:
18  • Data pipeline: transformation, filtering, deduplication, mixing
19  • Mid-training + SFT: synthetic data
20
21  # Data pipeline
22  transformation()
23  filtering()
24  deduplication()
25  data_mixing()
26
27  # Post-training data
28  post_training_data()
29
30  Summary:
31  • Filtering: train classifier (language id, quality, toxicity) for what good looks like
32  • Deduplication: hashing scales to large datasets for fuzzy matching
33  • Mixing: train mixtures at small scale, extrapolate to optimal mixtures and large scale
```

图 1: 本讲路线：转换、过滤、去重、混合，以及 SFT/合成数据。

核心观点

数据工程不是把文本拼接起来；每个变换都在重写模型将看到的经验分布。处理代码、阈值和随机种子与模型超参数同等重要。

1.1 本章小结

最终语料应是可追溯的 token 流，而不是来源不明的文本目录。

2 转换：把复杂文档还原成阅读顺序

HTML、PDF 与代码都不是纯文本。PDF 尤其同时包含字形位置、字体、图像与多栏布局；错误阅读顺序会制造看似流畅却语义错位的样本。

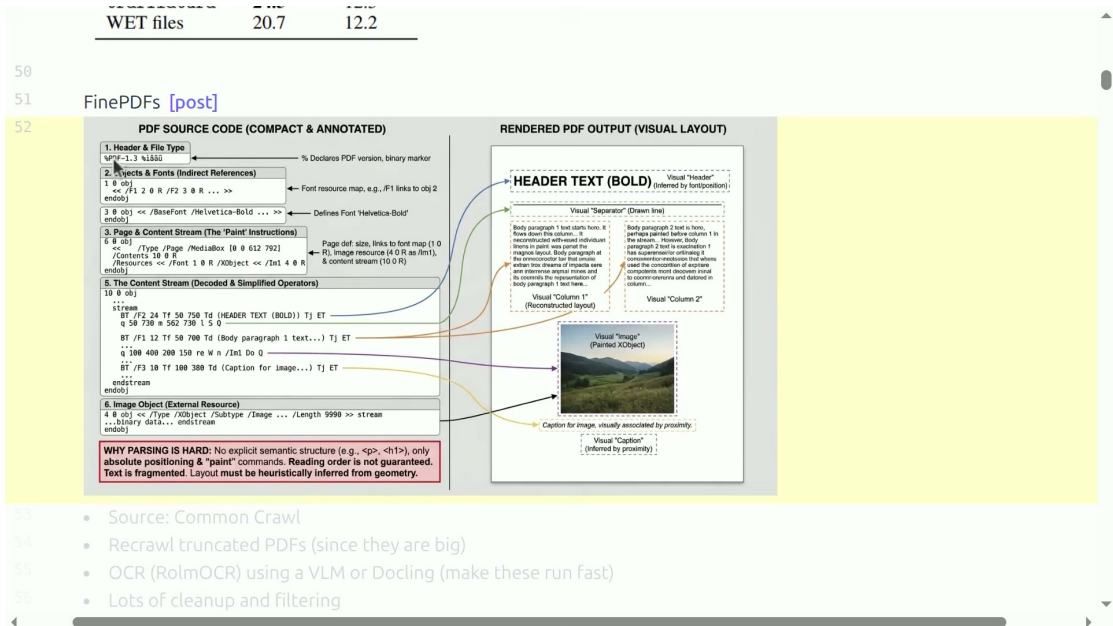


图 2: PDF 源码对象与视觉布局的映射；转换器必须恢复标题、段落、表格与阅读顺序。

转换审计

保存原文件哈希、解析器版本、版面框、OCR 置信度与失败原因；抽样比较原页面和提取文本，而非只统计成功率。

2.1 本章小结

转换质量决定后续过滤器看到什么，解析错误无法靠增加数据量自动修复。

3 过滤：数量与质量的非单调关系

规则、语言识别器与质量模型会减少 token，却可能提升固定计算下的学习效率。过滤过强则丢失多样性；高质量小集合反复多 epoch 还会过拟合。



图 3: 不同过滤配方的损失随训练 token 变化：质量收益与数据耗尽风险同时存在。

若文档质量分数为 $q(d)$ ，保留规则为 $I[q(d) \geq \tau]$ 。其中 q 是分类器或规则聚合， τ 是阈值， I 为指示函数。阈值必须通过固定模型、固定训练 token 的消融选择。

代理偏差

“像百科”“教育价值高”等标签只是代理目标；应按语言、域和文体报告通过率，防止过滤器把少数表达当作噪声。

3.1 本章小结

最优过滤器取决于训练预算，不存在脱离模型规模与 epoch 数的普适阈值。

4 重复：从完全相同到模板近似

完全重复包括镜像站与 fork；近重复包括轻微改写、格式变化、模板页与同一文章的不同版本。

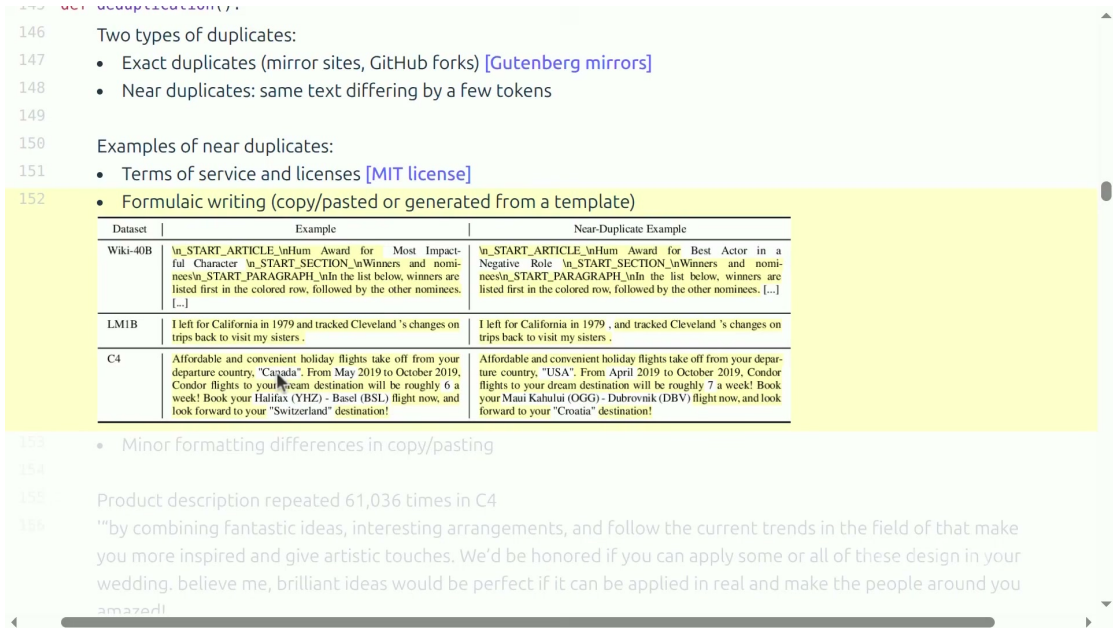


图 4：近重复示例：文本主体相同，仅表面格式或少量 token 不同。

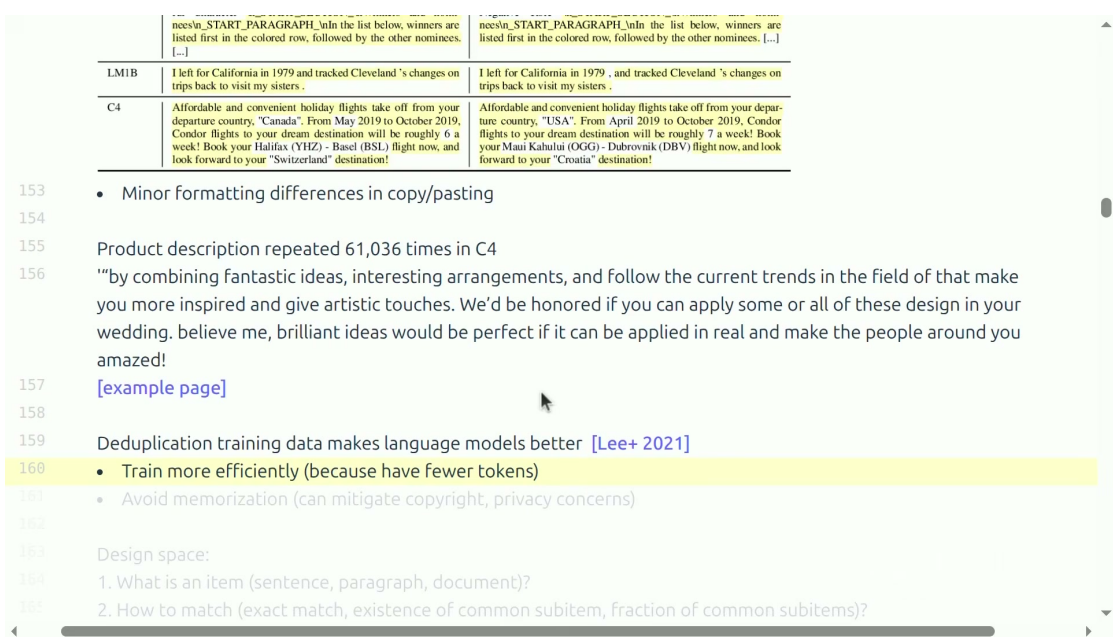


图 5：去重减少无效 token 与记忆风险，但需选择条目粒度和匹配规则。

去重目标

提高有效信息密度、降低训练浪费、减少测试污染与逐字记忆；同时避免误删模板相似但事实不同的记录。

4.1 本章小结

去重不是布尔操作，而是条目定义、相似度、阈值和代表样本选择的联合设计。

5 精确去重与哈希分组

标准化文本后计算内容哈希，将同哈希项目排序分组，每组保留一个代表。它语义清晰、精度高、易并行，但不能发现近重复。

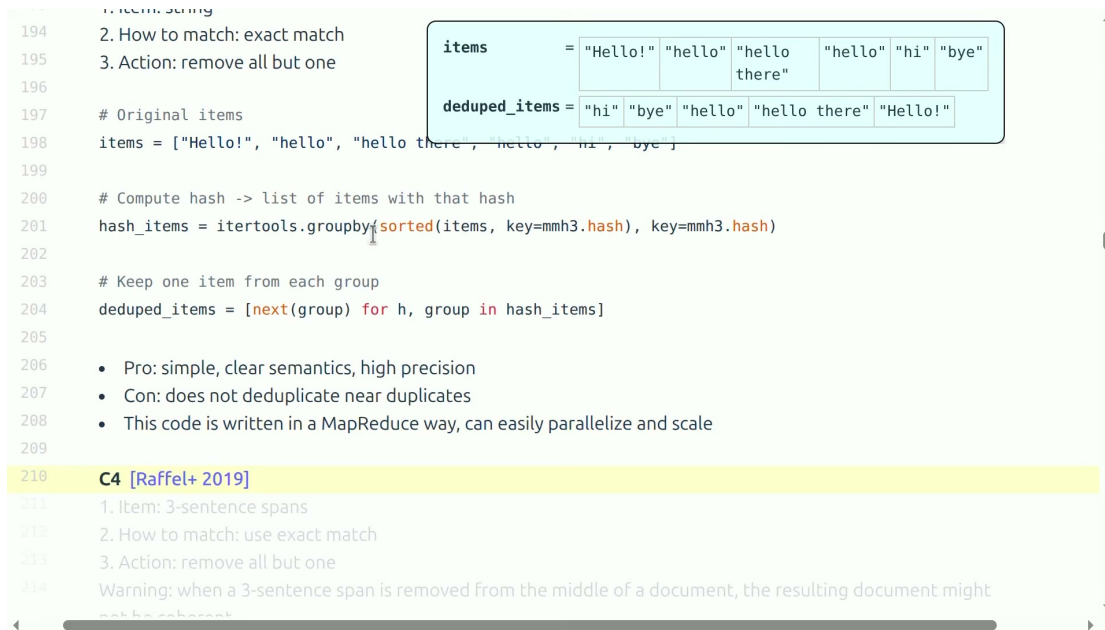


图 6: 精确哈希去重：相同内容分组后每组保留一个项目。

Listing 1: 精确去重的流式实现

```
1 def exact_dedup(records):
2     seen = set()
3     for r in records:
4         key = sha256(normalize(r.text))
5         if key in seen:
6             continue
7         seen.add(key)
8         yield r
```

规范化边界

大小写、空白、Unicode 与标点规范化越激进，召回越高但误合并风险越大；规范化函数必须版本化。

5.1 本章小结

精确去重适合作为第一层，近重复仍需集合相似度与局部敏感哈希。

6 MinHash: 近似 Jaccard 相似度

把文档表示为 shingles 集合 $S(d)$, Jaccard 相似度为

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

A, B 是两篇文档的 shingle 集合, 分子为共有片段数, 分母为并集片段数。

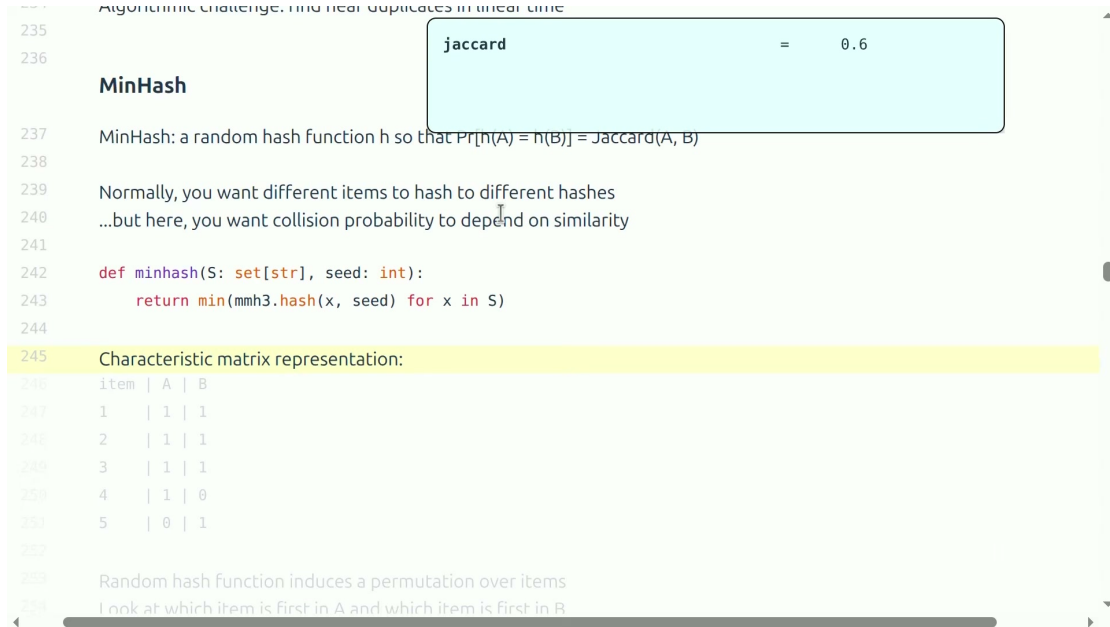


图 7: MinHash 的关键性质: 随机最小哈希相等的概率等于 Jaccard 相似度。

对 m 个独立哈希, 估计量

$$\hat{J} = \frac{1}{m} \sum_{k=1}^m I[h_k(A) = h_k(B)]$$

中, m 是签名长度, h_k 是第 k 个 MinHash, I 是指示函数。 m 越大, 方差越小但存储和比较更贵。

6.1 本章小结

MinHash 把大集合变成短签名, 使相似度估计可扩展到海量文档。

7 LSH: 只生成可能相似的候选对

全量两两比较是 $O(N^2)$ 。LSH 将签名分成 b 个 band、每 band r 行；至少一个 band 完全相同就成为候选。

```
258 # Verify MinHash approximates Jaccard as advertised
259 n = 100 # Generate this many random hash functions
260 matches = [minhash(A, seed) == minhash(B, seed) for seed in range(n)]
261 estimated_jaccard = len([m for m in matches if m]) / len(matches)
262 assert abs(estimated_jaccard - jaccard) < 0.01
263
264 Now we can hash our items, but a collision doesn't tell us Jaccard(A, B) > threshold.
265
266
267 def locality_sensitive_hashing():
268     Locality sensitive hashing (LSH) [book chapter]
269
270     Suppose we hash examples with just one MinHash function
271     P[A and B collide] = Jaccard(A, B)
272
273     On average, more similar items will collide, but very stochastic...
274
275     Goal: have A and B collide if Jaccard(A, B) > threshold
276     We have to somehow sharpen the probabilities...
277
278     Solution: use n hash functions
279     Partition into b bands of r hash functions each (n = b * r)
```

图 8: LSH 用多个哈希带把“相似即更可能碰撞”转成候选生成机制。

碰撞概率为

$$P_{\text{cand}}(s) = 1 - (1 - s^r)^b,$$

其中 s 是真实 Jaccard，相似度 s^r 是单个 band 相同概率， b 次独立尝试至少成功一次得到最终概率。

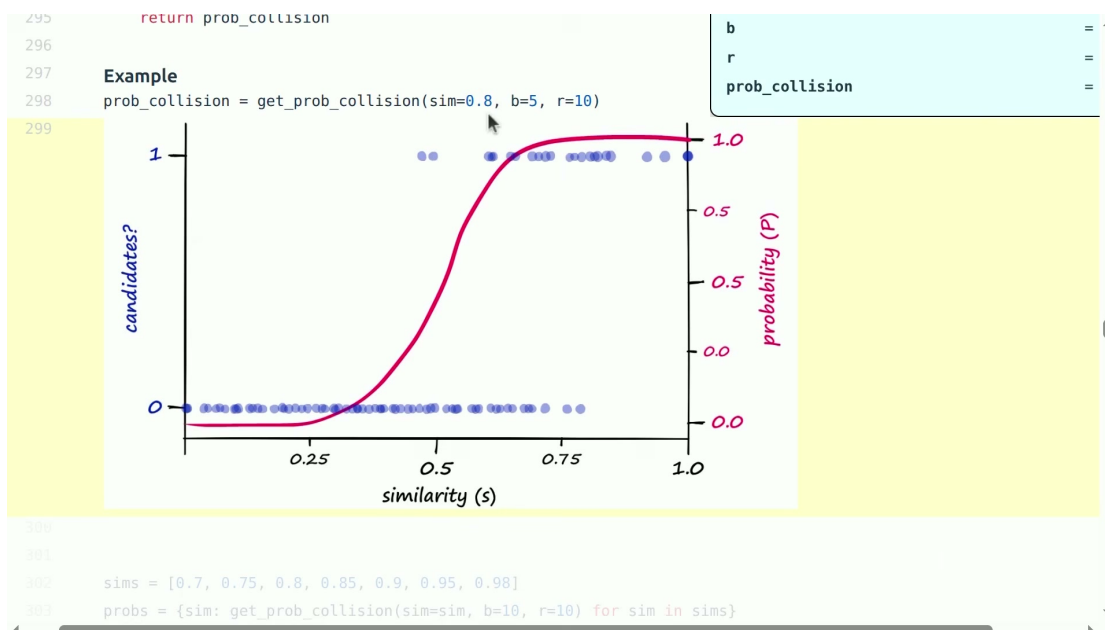


图 9: LSH 的 S 形碰撞曲线：参数决定相似度阈值附近的相变。

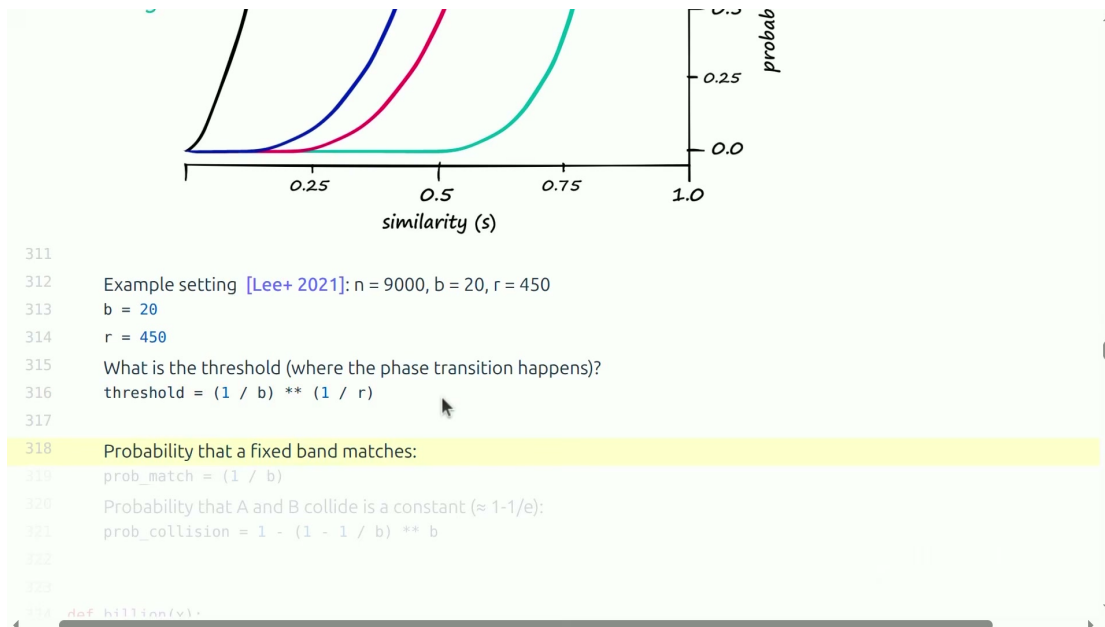


图 10: 近似阈值约为 $(1/b)^{1/r}$; 增加 band 提升召回, 增加行数提升精度。

7.1 本章小结

LSH 是候选生成器, 最终候选仍应计算精确相似度并记录去重簇。

8 数据混合：训练分布是一组权重

设来源 i 有 N_i token，混合权重为 p_i ，满足 $p_i \geq 0, \sum_i p_i = 1$ 。按规模采样令 $p_i = N_i / \sum_j N_j$ ；均匀采样给小来源更高相对权重。

337 The Pile [Gao+ 2020]

338

Component	Raw Size	Weight	Epochs	Effective Size	Mean Document Size
Pile-CC	227.12 GiB	18.11%	1.0	227.12 GiB	4.33 KiB
PubMed Central	90.27 GiB	14.40%	2.0	180.55 GiB	30.55 KiB
Books3 [†]	100.96 GiB	12.07%	1.5	151.44 GiB	538.36 KiB
OpenWebText2	62.77 GiB	10.01%	2.0	125.54 GiB	3.85 KiB
ArXiv	56.21 GiB	8.96%	2.0	112.42 GiB	46.61 KiB
Github	95.16 GiB	7.59%	1.0	95.16 GiB	5.25 KiB
FreeLaw	51.15 GiB	6.12%	1.5	76.73 GiB	15.06 KiB
Stack Exchange	32.20 GiB	5.13%	2.0	64.39 GiB	2.16 KiB
USPTO Backgrounds	22.90 GiB	3.65%	2.0	45.81 GiB	4.08 KiB
PubMed Abstracts	19.26 GiB	3.07%	2.0	38.53 GiB	1.30 KiB
Gutenberg (PG-19) [†]	10.88 GiB	2.17%	2.5	27.19 GiB	398.73 KiB
OpenSubtitles [†]	12.98 GiB	1.55%	1.5	19.47 GiB	30.48 KiB
Wikipedia (en) [†]	6.38 GiB	1.53%	3.0	19.13 GiB	1.11 KiB
DM Mathematics [†]	7.75 GiB	1.24%	2.0	15.49 GiB	8.00 KiB
Ubuntu IRC	5.52 GiB	0.88%	2.0	11.03 GiB	545.48 KiB
BookCorpus2	6.30 GiB	0.75%	1.5	9.45 GiB	369.87 KiB
EuroParl [†]	4.59 GiB	0.73%	2.0	9.17 GiB	68.87 KiB
HackerNews	3.90 GiB	0.62%	2.0	7.80 GiB	4.92 KiB
YoutubeSubtitles	3.73 GiB	0.60%	2.0	7.47 GiB	22.55 KiB
PhilPapers	2.38 GiB	0.38%	2.0	4.76 GiB	73.37 KiB
NIH ExPorter	1.89 GiB	0.30%	2.0	3.79 GiB	2.11 KiB
Enron Emails [†]	0.88 GiB	0.14%	2.0	1.76 GiB	1.78 KiB
The Pile	825.18 GiB			1254.20 GiB	5.91 KiB

339 Key question: what distribution over the data sources should we use?

图 11: 不同数据来源规模差异巨大；混合权重决定模型把多少更新分配给每个域。

Listing 2: 按来源概率构造训练流

```
1 while tokens_seen < budget:
2     i = rng.choice(num_sources, p=mixture)
3     batch = sources[i].next_batch()
4     train_step(batch)
5     audit.add(source=i, tokens=batch.num_tokens)
```

8.1 本章小结

混合权重是模型超参数，必须随 checkpoint 发布并按实际消费 token 审计。

9 epoching: 小而优的数据也会被耗尽

来源 i 的期望 epoch 数为

$$E_i = \frac{p_i T}{N_i},$$

其中 T 是总训练 token, $p_i T$ 是该来源被消费的 token, N_i 是其独立 token 规模。

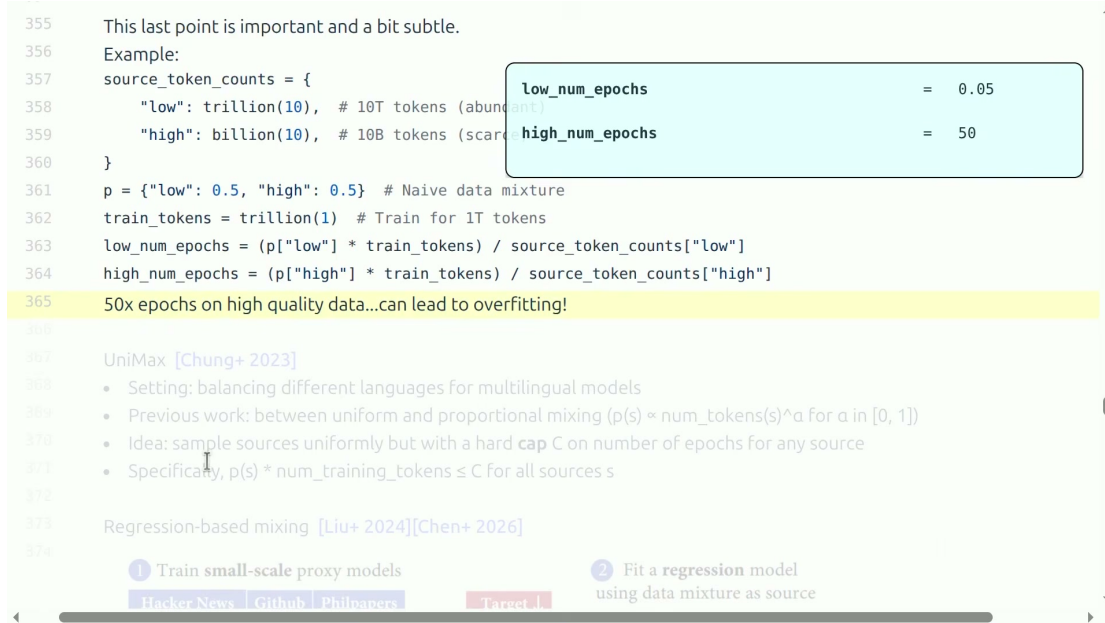


图 12: 相同混合概率可让小来源重复 50 epoch, 而大来源只见到 0.05 epoch。

高质量不等于可无限重复

过多 epoch 会让模型记忆样本并让代理实验误判某来源价值。可设置每来源 epoch 上限或使用随训练阶段变化的配比。

9.1 本章小结

混合优化必须同时考虑质量、规模和重复次数。

10 回归式混合：用代理模型搜索配方

先训练多个小模型覆盖不同混合点，再用回归模型预测下游损失，最后在预测曲面上优化大模型配方。

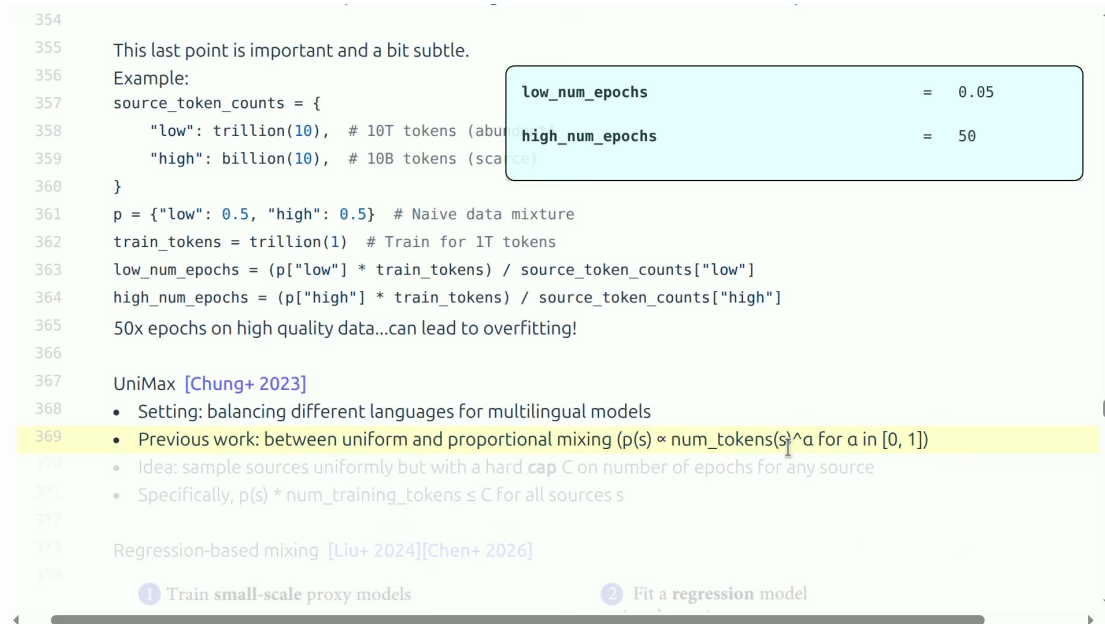


图 13: 回归式混合的四步：代理训练、拟合、模拟搜索、用最佳配方训练大模型。

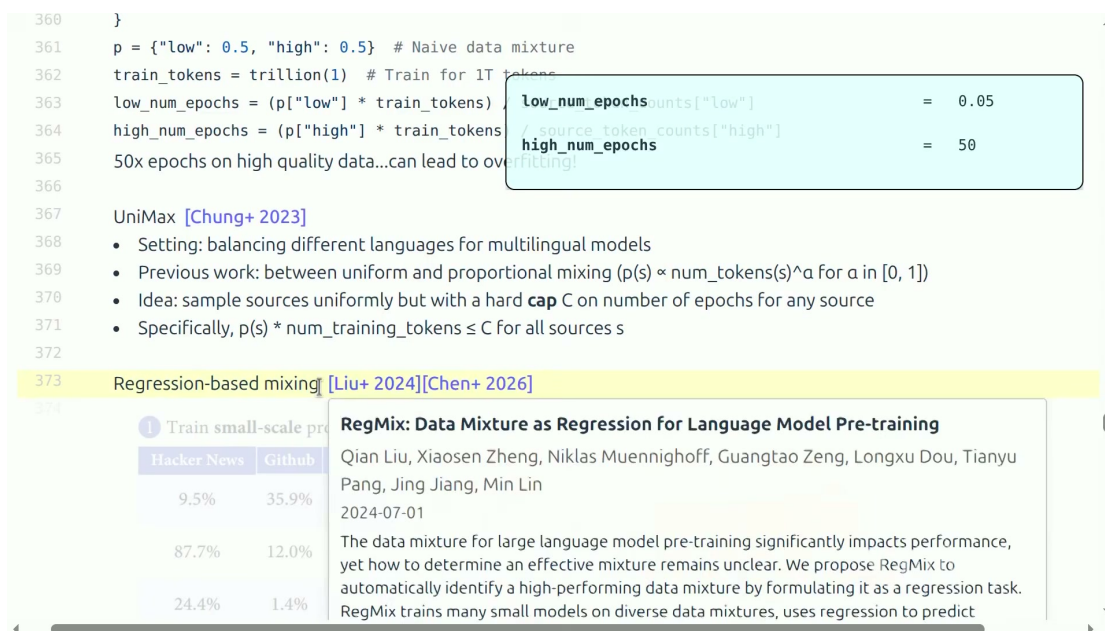


图 14: 混合比例到目标损失的响应曲面；最优点依赖目标任务定义。

若代理观测为 $(p^{(k)}, y^{(k)})$ ，拟合 $\hat{y} = f_{\phi}(p)$ 并求 $p^* = \arg \min_{p \in \Delta} \hat{y}$ 。 Δ 是概率单纯形， ϕ 为回归参数。代理到大模型的迁移误差必须用少量大规模验证点校准。

10.1 本章小结

数据混合可像 scaling law 一样建模，但代理规模、目标任务和 epoching 会造成系统偏差。

11 合成数据：从环境和教师收集响应

配方是：定义环境与任务，从强教师模型收集高质量轨迹，再做验证、过滤和去重。

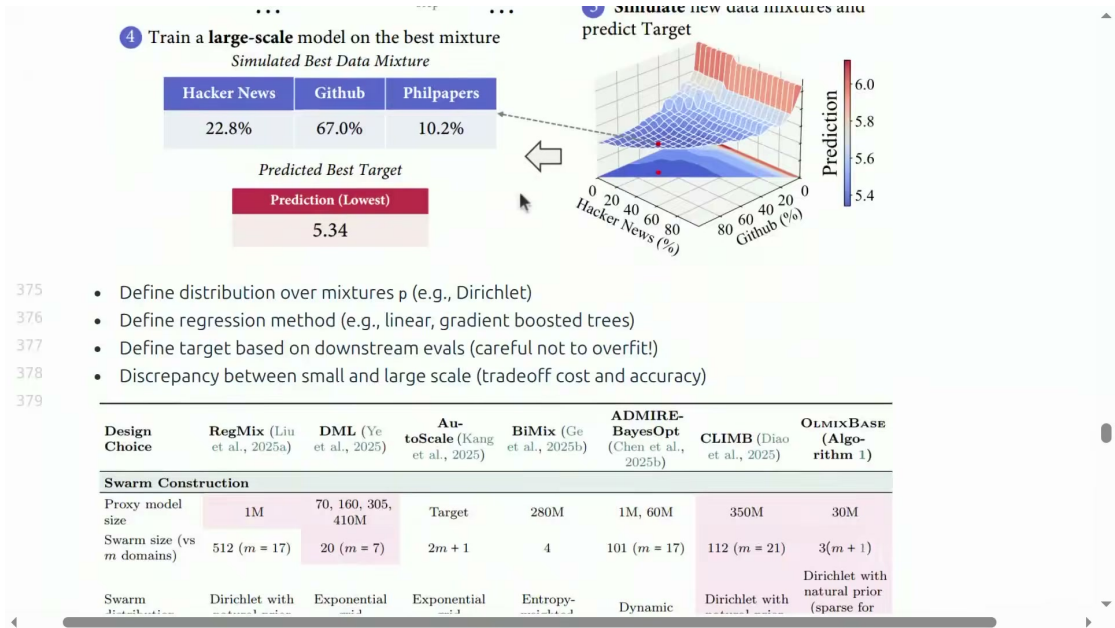


图 15: 后训练数据配方：定义环境、任务/提示，再从教师模型采集响应。

教师数据的最小记录

教师模型与版本、system prompt、采样温度、工具环境、输入任务、原始响应、验证器结果和过滤理由。

11.1 本章小结

合成数据把模型变成数据生成器，但质量由任务分布和验证器而非流畅度决定。

12 代码智能体任务构造

SWE-smith 从真实仓库创建环境，让模型生成可验证任务；SWE-rebench 从 GitHub PR 出发，经仓库安装、LLM 标注与测试验证形成实例。

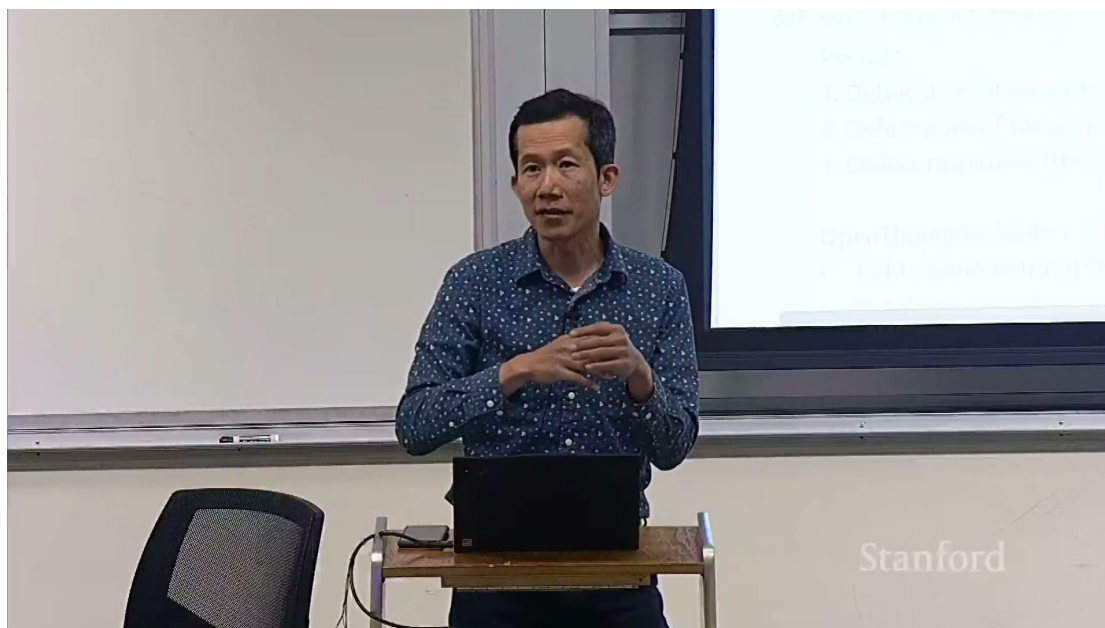


图 16: SWE-smith：仓库、环境、任务生成策略与验证共同形成新任务实例。

```
399 • In this downsampled mixture, models that epoch too much won't look good.
400 • So the optimum will be more balanced.
401 p = {"low": 0.7, "high": 0.3} # More mass on high quality data
402
403 Summary:
404 • Problem: how to weight different data sources (e.g., Wikipedia, general, code)
405 • Regression-based mixing: estimate mixture → loss at small scale, optimize (analogous to scaling laws)
406 • Important consideration: epoching and overfitting (solution: cap or simulated)

409 def post_training_data():
410     Recipe:
411     1. Define a set of environments
412     2. Define a set of tasks / prompts
413     3. Collect responses from a strong model! (teacher)

OpenThoughts [Guha+ 2025]
• 1.2M examples using QwQ-32B as a teacher
• Questions come from 27 human and synthetic sources (e.g., StackExchange, NuminaMath, Chemistry)
  • StackExchange CodeGolf (Number of Questions: 85.9K): A StackExchange forum of coding puzzles, specifically aimed at solutions with the least number of characters possible.
  • OpenCodeReasoning (Number of Questions: 459K): A large reasoning-based synthetic dataset to date for codeint - comprises 735,755 samples in Python across 78,119 unique
```

图 17: SWE-rebench：从 PR 元数据到可安装环境、LLM 标注与最终数据集。

可执行验证优先

代码题的价值来自能安装、能运行、能由测试判定；只生成自然语言描述会混入不可解或答案泄漏任务。

12.1 本章小结

环境构建是 agent 数据集的核心，任务文本只是其可见表面。

13 教师数据的偏差与蒸馏边界

教师模型会把自身风格、错误与安全边界传给学生；重复采样还会降低多样性。应混合多教师、保留原始来源、使用规则或执行器验证，并对教师—学生重叠做消融。

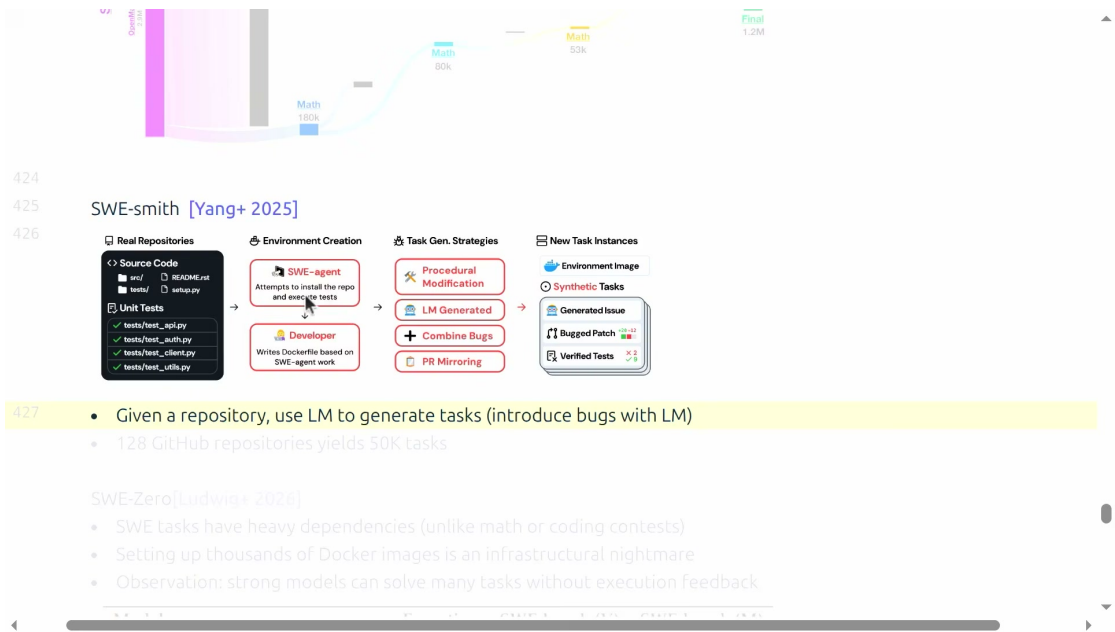


图 18: 教师响应进入训练前仍需环境验证、质量控制和分布审计。

循环污染

用同一家族模型生成、筛选并评估数据，可能形成闭环自证。至少保留独立评测器与真实数据锚点。

13.1 本章小结

蒸馏压缩教师行为，不自动创造教师之外的事实或可靠性。

14 端到端数据协议

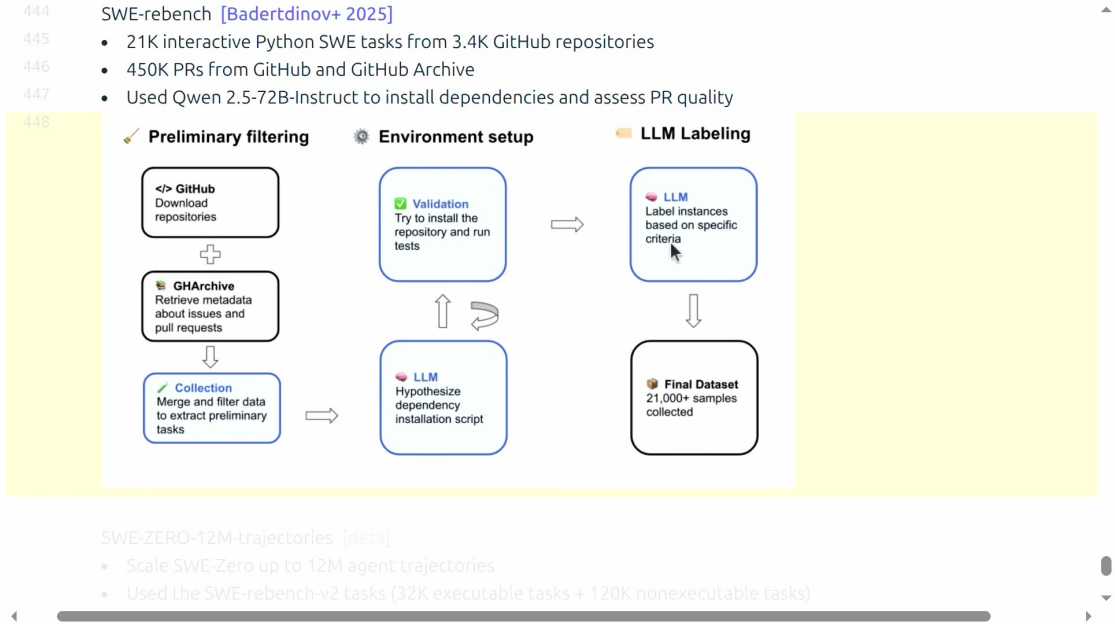


图 19: 现代数据管线把采集、过滤、去重、混合与合成任务组织成可验证阶段。

1. 冻结原始对象与来源许可；
2. 版本化转换和过滤；
3. 精确去重后做 MinHash/LSH；
4. 记录去重簇和代表选择；
5. 用代理模型搜索混合；
6. 限制各来源 epoch；
7. 对合成任务做可执行验证；
8. 用固定训练预算完成端到端消融。

Listing 3: 可复现的数据阶段接口

```
1 artifact = run_stage(  
2     input_manifest="raw-v3.jsonl",  
3     code_commit="abc123",  
4     config="dedup_lsh.yaml",  
5     seed=2026,  
6 )  
7 assert artifact.counts.windows == artifact.audit.counts
```

14.1 本章小结

每一步既输出数据，也输出计数、原因、哈希与版本元数据。

15 总结与延伸

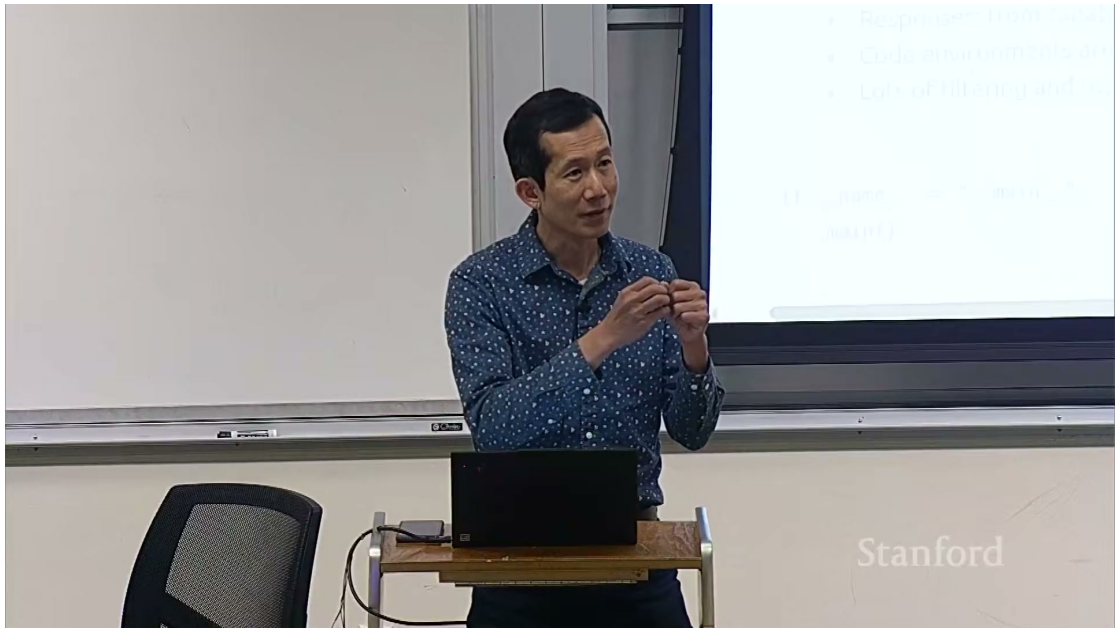


图 20: 本讲结尾回到核心：转换、过滤、去重、混合与合成数据共同决定训练经验。

最终结论

高价值数据不是“更干净文本”的同义词，而是有限训练预算下，能以可追溯方式提高目标能力且不放大记忆、偏见和合规风险的数据。

15.1 进一步推论

数据价值随模型规模和训练阶段变化；去重阈值与混合权重应进入 scaling 实验；合成数据最终会把瓶颈从“生成多少”移到“如何验证”。

15.2 建议练习

1. 实现 MinHash+LSH 并画出不同 b, r 的碰撞曲线；
2. 比较文档级和段落级去重的误删；
3. 固定 token 预算搜索三来源混合；
4. 对小来源加入 epoch cap；
5. 为代码任务构建可重放容器和测试判定器。